File-by-File Guide - What Each File Does

Quick reference for interview: 2-3 line summary of what each file in the project does.

Root Files

`README.md`

Professional project overview with features, tech stack, and setup instructions. First thing anyone sees on GitHub - explains what the app is and how to run it.

`.gitignore`

Tells Git which files to ignore (node_modules, .env, build outputs). Prevents accidentally committing sensitive data or large dependencies.

`vercel.json`

Deployment configuration for Vercel (frontend hosting). Specifies build command, output directory, and environment variables for production deployment.

`render.yaml`

Deployment configuration for Render/Railway (backend hosting). Defines how to build and run the Node.js server in production.

Client Folder (`client/`) - Frontend

Configuration Files

`package.json` Lists all frontend dependencies (React, Socket.io-client, Tailwind) and npm scripts (dev, build). Defines project metadata and what libraries the frontend uses.

`vite.config.js` Vite dev server configuration - enables ngrok tunneling with allowedHosts and proxies Socket.io requests to backend. Critical for making the demo work with ngrok.

`tailwind.config.js` Tailwind CSS configuration - defines colors, spacing, and styling system. Makes the UI look clean and modern with utility classes.

`postcss.config.js` PostCSS configuration for processing Tailwind CSS. Required for Tailwind to work with Vite build system.

`.oxlintrc.json` Linter configuration for catching code errors and enforcing style. Helps maintain clean, consistent code.

Client Source (`client/src/`)

`main.jsx` Entry point for React app - renders App component into DOM. First file that runs when the frontend starts.

`App.jsx` Creates Socket.io connection and sets up React Router (Lobby vs Room pages). Manages global socket instance and navigation between create/join and watch party room.

Components (`client/src/components/`)

`Lobby.jsx` Landing page where users create or join rooms. Has two forms: create new room (generates random code) or join existing room (enter code).

`Room.jsx` Main watch party UI - combines VideoPlayer, Chat, Playlist, MovieBrowser, and Participants. The "control center" that arranges all features in a grid layout.

`VideoPlayer.jsx` Embeds YouTube IFrame player and handles play/pause/seek user interactions. Displays the actual video that everyone watches together.

`Chat.jsx` Live chat component with message input, message list, and auto-scroll. Lets users talk to each other in real-time during the watch party.

`MovieBrowser.jsx` Search and browse movies using TMDB API - shows trending, popular, and search results. Lets users find YouTube trailers to load into the player.

`Playlist.jsx` Queue of videos with play/remove buttons - shows what's up next. Users can add multiple videos and play them in order.

`Reactions.jsx` Floating emoji animations triggered by users clicking reaction buttons. Shows 😂❤️👍 emojis floating up the screen when someone reacts.

`ConnectionStatus.jsx` Displays "Connected" or "Reconnecting..." indicator in UI. Shows users if their Socket.io connection is working or having issues.

`Polls.jsx` (Not currently used in main UI) - Simple yes/no poll component. Was removed from main UI but code still exists.

Hooks (`client/src/hooks/`)

`useRoomSocket.js` Custom hook that manages Socket.io connection and provides commands (sendPlay, sendPause, sendMessage). Centralizes all socket communication logic so components just call functions like `sendPlay()` .

`useSyncedPlayer.js` Custom hook that syncs video player with server state - handles play/pause/seek from server and runs drift correction. The brain of video synchronization - makes sure everyone stays in sync.

`useClockOffset.js` Measures time difference between client and server clocks using ping/pong. Needed to accurately calculate current video position from server timestamps.

`useConnectionStatus.js` Tracks Socket.io connection state (connected, disconnected, reconnecting). Used by ConnectionStatus component to show connection indicator.

Services & Utils (`client/src/services/` , `client/src/utils/`)

`utils/tmdb.js` Functions to fetch movie data from TMDB API (trending, popular, search). Provides `getTrending()` , `getPopular()` , `searchMovies()` for MovieBrowser component.

`utils/constants.js` Defines all Socket.io event names (PLAY, PAUSE, CHAT_MESSAGE, etc.) as constants. Ensures client and server use exact same event names - prevents typos.

Server Folder (`server/`) - Backend

Configuration

`package.json` Lists backend dependencies (Express, Socket.io, CORS) and start script. Defines what the Node.js server needs to run.

Server Source (`server/src/`)

`index.js` Entry point - creates Express server, sets up Socket.io, and starts listening on port 3001. Initializes the backend and connects all the pieces.

`socket.js` Socket.io event handlers - processes PLAY, PAUSE, JOIN_ROOM, CHAT_MESSAGE events from clients. The "brain" of the backend - handles all real-time commands and broadcasts updates.

`rooms.js` Room manager class - stores room state in memory and provides methods to create/update/delete rooms. Like a database for rooms but in RAM (lost on server restart).

`constants.js` Defines Socket.io event names (same as client constants.js). Keeps event names consistent between frontend and backend.

Scripts Folder (`scripts/`)

`loadtest.js` Performance testing script that simulates multiple users joining rooms and playing videos. Used to test if the app can handle 10+ concurrent users without lagging.

`package.json` Dependencies for load testing (socket.io-client). Separate from main app because it's just for testing.

Documentation (`docs/`)

`UNDERSTANDING.md` Comprehensive technical deep-dive explaining how the entire project works - tech stack, sync algorithm, code walkthrough. 1300+ lines of detailed explanation for understanding the architecture.

Quick Answer Guide (If They Ask)

"What does App.jsx do?" → Creates the Socket.io connection and sets up routing between the lobby (create/join room) and the actual watch party room.

"What's the difference between useRoomSocket and useSyncedPlayer?" → useRoomSocket handles communication (sending/receiving socket events), useSyncedPlayer handles video synchronization (keeping video in sync with server state).

"Where's the sync logic?" → Main sync algorithm is in `client/src/hooks/useSyncedPlayer.js` (client-side) and `server/src/socket.js` (server-

side event handlers).

"How do you prevent conflicts when two people click play at once?" → Version control in `server/src/rooms.js` - each state change increments a version number, server rejects commands with stale versions.

"Where's the video player?" → `client/src/components/VideoPlayer.jsx` - embeds YouTube IFrame API and handles user play/pause/seek actions.

"How does chat work?" → `Chat.jsx` component sends messages via socket (`CHAT_MESSAGE` event), `socket.js` broadcasts to all users in room, everyone's Chat component receives and displays.

"What's drift correction?" → In `useSyncedPlayer.js` - runs every 3 seconds, checks if your video position matches server's calculated position, snaps back if off by >2 seconds.

"How do you find movies?" → `MovieBrowser.jsx` uses TMDB API (via `tmdb.js` service) to search/browse movies and get YouTube trailer links.

"What's vercel.json for?" → Tells Vercel how to deploy the React frontend (build command, output directory, environment variables).

"What's render.yaml for?" → Tells Render/Railway how to deploy the Node.js backend (start command, environment variables, port).

Files NOT on GitHub (Study Materials)

These are in `/Users/sajivvaila/watch-party-study-docs/` locally (not pushed to GitHub):

- `START.md` - How to run the app
- `FEATURES.md` - Feature walkthrough
- `DESIGN.md` - Architecture deep dive
- `INTERVIEW.md` - Interview Q&A
- `INTERVIEW-DEMO-CHECKLIST.md` - ngrok demo steps
- `QUICK-START-DEMO.md` - Quick demo script

Why not on GitHub? Study materials are for you, not for the public repo. GitHub shows the clean project, not your interview prep notes.

Most Important Files to Know Well

1. **`client/src/hooks/useSyncedPlayer.js`** - The sync algorithm (heart of the project)

2. `server/src/socket.js` - Server-side event handling (how commands are processed)
3. `client/src/App.jsx` - Entry point and socket setup
4. `server/src/rooms.js` - Room state management with version control
5. `client/src/hooks/useRoomSocket.js` - Client-server communication

Know these 5 files deeply and you can explain the entire architecture!

You've got this! 🚀